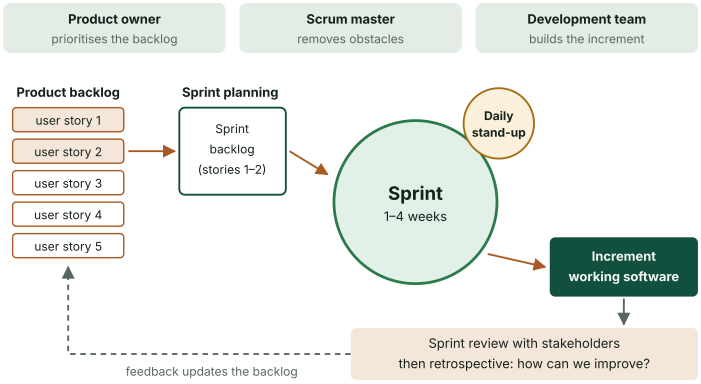
Prototyping, RAD and Agile methods

AP11 FACT FILE

Specification: students should be able to

- Describe prototyping, including throwaway and evolutionary prototyping
- Describe the main features and phases of Rapid Application Development (RAD), including user involvement
- Describe the main features of the Agile approach, including product and release backlogs, sprints, the Scrum team, daily Scrums and burn down charts
- Describe extreme programming (XP) practices such as pair programming and test-driven development
- Evaluate the waterfall model, RAD and Agile and recommend an approach for a given project

Key words: prototype, throwaway prototyping, evolutionary prototyping, rapid application development (RAD), user design, cutover, timeboxing, Agile, product backlog, release backlog, sprint, Scrum master, product owner, daily Scrum, burn down chart, sprint retrospective, user story, pair programming



The Scrum framework: a product backlog feeds sprint planning, the team works in a 1–4 week sprint with daily stand-ups, producing an increment that is reviewed

Overview: faster, more flexible approaches

The waterfall model delivers a system only at the end and involves users mainly at the start and during testing. Rapid Application Development and Agile methods were developed to overcome those weaknesses. Both rely on frequent, close contact with users and on producing working versions early, so that misunderstandings are found while they are still cheap to correct. RAD, developed by James Martin in the late 1980s and popularised by his 1991 book, delivers a system quickly through four phases, heavy use of prototypes, a fixed timescale and requirements ranked so that low-priority extras can be dropped. Agile, which takes its name from a manifesto published by a group of developers in 2001, delivers software in a series of short sprints, guided by a prioritised product backlog and organised by a multi-skilled Scrum team led by a Scrum master. Prototyping, in either its throwaway or evolutionary form, is a technique used within these approaches. None of these methods is always best, so you must be able to evaluate each one against the needs of a particular project.

Prototyping

A prototype is an early model of all or part of a system, built quickly so that users can see it and try it out. It is often a mock-up with little real processing behind it, but it shows the screens, menus and reports that users will work with. This helps users to identify and clarify requirements that they find difficult to describe in words, and it is particularly useful for testing alternative user interfaces. In throwaway prototyping the model is discarded once the requirements have been agreed, and the real system is then built properly using production tools and standards. In evolutionary prototyping the prototype is refined over and over in response to feedback until it becomes the final system, so none of the effort is wasted. The risks are different. A throwaway prototype can make users think the system is nearly finished, while an evolutionary prototype may carry poor structure from early, rushed decisions into the final product, making it harder to maintain and encouraging scope creep.

Rapid Application Development

RAD splits a project into a series of builds and sets a fixed time limit that must not be exceeded. Requirements are put in priority order, so if time runs short the desirable add-ons are sacrificed rather than the deadline. There are four phases. In requirements planning, everyone with an interest meets to agree the scope and key requirements; unlike waterfall, the requirements do not have to be fully fixed, because changes are expected. In user design, users and analysts work closely together, using prototypes to design the functions and to compare alternative interfaces. In construction, the programs are written but are continually refined as users comment on real screens and reports, and the modules are integrated and tested. In cutover, the new system replaces the old, which involves further testing, data conversion and training. RAD gets a system into use much sooner and keeps users involved throughout, so errors surface early. However, it needs skilled practitioners trained in the method and committed users, it can be expensive, and it suits large, safety-critical systems poorly.

Agile: backlogs, sprints and the Scrum team

An Agile project begins with a product backlog, an initial prioritised wish list of everything the product should do. At the start of each phase, release planning selects features from the product backlog and moves them into a release backlog. These are broken into work items, which may be new features, improvements or investigations, then estimated and ordered by priority. The work is carried out in sprints, fixed periods of a few weeks, each delivering a working increment, so the product grows incrementally. Scrum is the framework for running a project in sprints. The Scrum team is small and multi-skilled, typically including developers, testers and customer input, so it can analyse, design, build and test, and everyone contributes to discussion. The Scrum master makes sure the team works effectively, arranges meetings, monitors progress and removes obstacles, much like a project manager. The product owner is the voice of the client, setting priorities in the backlogs and preventing unrealistic items from being added. Each sprint ends with a sprint review of the product and a retrospective on the process.

Monitoring a sprint

Close monitoring keeps an Agile project on schedule. Each day begins with a daily Scrum, a short meeting of about fifteen minutes, often held standing up to keep it brief. Every member answers three questions: what they did yesterday, what they plan to do today and what obstacles are in their way. The Scrum master takes responsibility for resolving the obstacles raised, although everyone may contribute ideas. Progress is shown on a burn down chart, which plots the work remaining against the days of the sprint. The ideal line falls in a straight line from all the work at the start to none at the end. For example, with 60 story points in a 10-day sprint the ideal rate is 6 points per day. If the actual line sits above the ideal line, the team is behind schedule; if it is below, the team is ahead. The chart lets the Scrum master spot a sprint at risk early and take corrective action, such as moving a multi-skilled team member onto a task that is overrunning.

Evaluating Agile and extreme programming

Agile has clear advantages. A project can start quickly with the most important requirements and then evolve, working deliverables appear early, continuous contact with the customer makes misunderstandings less likely, and multi-skilled team members can be moved to whichever task is falling behind. Its weaknesses are that the time needed for tasks can be difficult to estimate, how good those estimates are relies heavily on how experienced the Scrum master is, documentation may be less detailed than in traditional methods, and the customer must commit time throughout, which makes a fixed price and scope hard to agree in advance. Extreme programming is an Agile method that concentrates on engineering practices to keep code quality high despite rapid change. In pair programming two developers share one workstation, one writing code and the other reviewing it, so errors are spotted immediately and knowledge is shared. Test-driven development writes automated tests before the code. Small, frequent releases, continuous integration, refactoring and an on-site customer complete the approach.

T Key terms from the mark schemes

Prototype
An early model of all or part of a system, built quickly so that users can evaluate it and clarify their requirements.
`Credited: Mock-up`

Throwaway prototyping
An approach in which a prototype is used to establish requirements and is then discarded before the real system is built.

Evolutionary prototyping
An approach in which a prototype is repeatedly refined in response to user feedback until it becomes the final system.

Rapid application development (RAD)
A methodology that delivers systems quickly through four phases (requirements, user design, construction, cutover), frequent prototypes, close user involvement and a strict timescale with prioritised requirements.
Credited: RAD

Timeboxing
Fixing the time allowed for a project or part of it so that the deadline cannot be overrun; lower-priority features are dropped instead.
Credited: Timebox

Product backlog
In Agile, the prioritised wish list of all the features the product should eventually provide.

Release backlog
The features chosen from the product backlog for the next release, broken down into work items.

Sprint
In Agile, a fixed period of a few weeks in which the team produces a working increment of the product.
Credited: Iteration

Scrum master
The team member who ensures the Scrum team works effectively, arranges meetings, monitors progress and removes obstacles.

Daily Scrum
A short daily meeting of about 15 minutes in which each team member says what they did yesterday, what they will do today and what obstacles they face.
Credited: Daily stand-up

Burn down chart
A graph of the work remaining in a sprint against time, compared with an ideal straight line, used to check whether work is on schedule.
Credited: Burndown chart

Pair programming
An extreme programming practice in which two programmers work at one computer, one writing code and the other reviewing it, swapping roles regularly.

C C# example

Worked example: a burn down chart

```
int total = 60, days = 10;
int[] actual = { 60, 53, 48, 44, 40, 34, 30, 22, 14, 7, 0 };

for (int day = 0; day <= days; day++)
{
    int ideal = total - (total / days) * day;
    string status = actual[day] > ideal ? "behind" : "on track";
    Console.WriteLine($"Day {day}: ideal {ideal}, actual {actual[day]} ({status})");
}
```

Exam focus

Typical questions ask you to describe the main features of RAD or Agile, briefly describe each RAD phase, describe user involvement, or give two advantages and two disadvantages, usually at one or two marks per point. For RAD, credit is given for builds, a strict timescale, requirements ranked in priority with desirable add-ons dropped, frequent prototypes and the four named phases: requirements, user design, construction and cutover. For Agile, name the product backlog, the release backlog, sprints and the Scrum master co-ordinating the team. User involvement should be stated precisely: continual, through prototypes in RAD or constant consultation in Agile. Distinguish throwaway from evolutionary prototyping by what happens to the prototype afterwards. Do not confuse the sprint review, which examines the product, with the retrospective, which examines the process. In extended evaluation questions, compare all three approaches against the scenario – stability of requirements, user availability, deadline, size and need for documentation – and finish with a justified recommendation. Generic lists of features without reference to the context score poorly.

? Check yourself

1. Explain the difference between throwaway and evolutionary prototyping. [2]
2. A Craigavon developer promises a hotel group a working room-booking system 'in twelve weeks, guaranteed' using Rapid Application Development (RAD). Describe the features of RAD that make such a promise possible. [4]
3. Name the four phases of RAD and briefly describe what happens in the user design phase. [4]

Answers: 1. Throwaway: the prototype is discarded once requirements are agreed and the real system is built separately (1) / Evolutionary: the prototype is refined repeatedly until it becomes the final system (1). 2. Any four from: project divided into a series of builds (1) / strict timescale/timebox that cannot be overrun (1) / requirements ranked in priority order (1) / desirable add-ons dropped if time runs out (1) / prototypes produced frequently and delivered to users for feedback (1) / much more user involvement than waterfall (1) / four phases: requirements, user design, construction, cutover (1) / reuse of components / GUI builders (1). 3. Requirements (planning), user design, construction, cutover (2 marks for all four in any order; 1 mark for two or three) / User design: users and analysts interact regularly (1) / prototypes/mock-ups show users what the system will look like and test alternative interfaces (1).